



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ingeniería Industrial y de Sistemas

ICS2121 Métodos de Optimización

Tarea 1 - Grupo 21

Integrantes:

- Francisca Isidora Ortiz Rojo
- Vicente Renato Jorge Olea Concha
- Santiago Ignacio García Lozano
- Franco Nicolai Corvalán Mansourova

Fecha de entrega: 3 de septiembre de 2025

Índice de contenido

1 Problema 1	3
1.1 Ítem a.	3
1.2 Ítem b.	4
1.3 Ítem c.	4
1.4 Ítem d.	5
2 Problema 2	6
3 Problema 3	8
4 Problema 4	12
4.1 Ítem a.	12
4.2 Ítem b.	13
4.3 ítem c.	14
5 Problema 5	15
5.1 Ítem a.	15
5.2 Ítem b.	16
6 Problema 6	18
6.1 Ítem a.	18
6.2 Ítem b.	19
6.3 Ítem c.	20
7 Problema 7	22
7.1 Ítem a.	22
7.1.1 Procedimiento subdiferencial $\partial g(x)$	22
7.1.2 Procedimiento subdiferencial $\partial h(x)$	23
7.1.3 Conclusión	24

7.2 Ítem b.	24
8 Referencias bibliográficas	26

1. Problema 1

1.1. Ítem a.

Para que la función quede igual a la del enunciado, es decir:

$$f(x) = \frac{1}{2}x^T Qx + c^T x$$

Se eliminó de las funciones del código perteneciente a `steepest.py`, cada parte en la que se mencionara el componente α o $\alpha(5 - x_n)^4$. Esto se logró cambiando las siguientes líneas (con sus respectivos números de línea):

```
# 1. ORIGINAL
51. funcion_objetivo = 0.5 * np.dot(np.transpose(x), np.dot(Q,x)) + np.dot(np.transpose(c), x) +
    alpha*(5 - x[n-1])**4

# 1. MODIFICADO
43. funcion_objetivo = 0.5 * np.dot(np.transpose(x), np.dot(Q,x)) + np.dot(np.transpose(c), x)

# 2. ORIGINAL
52. gradiente = np.dot(Q, x) + c + vector_canonico * (-4*alpha*(5-x[n-1])**3)

# 2. MODIFICADO
44. gradiente = np.dot(Q, x) + c

# 3. ORIGINAL
56. def funcion_enunciado(lambda_, Q, c, x, alpha, direccion_descenso):

# 3. MODIFICADO
48. def funcion_enunciado(lambda_, Q, c, x, direccion_descenso):

# 4. ORIGINAL
65. return (0.5 * np.dot(np.transpose(x), np.dot(Q,x)) + np.dot(np.transpose(c), x) + alpha*(5 -
    x[n-1])**4)[0][0]

# 4. MODIFICADO
55. return (0.5 * np.dot(np.transpose(x), np.dot(Q,x)) + np.dot(np.transpose(c), x))

# 5. ORIGINAL
120. lambda_ = scipy.optimize.fminbound(funcion_enunciado, 0, 10, args=(Q, c, x,
    direccion_descenso))

# 5. MODIFICADO
109. lambda_ = scipy.optimize.fminbound(funcion_enunciado, 0, 10, args=(Q, c, x,
    direccion_descenso))
```

1.2. Ítem b.

Se modificó el código del Método Simple de Primer Orden, y para ello, se editó el archivo FOMls.py y main_FOMls.py. En primer lugar, se presentan los cambios hechos en el archivo FOMls.py:

```
# 1. ORIGINAL
74. gradiente = np.matmul(np.transpose(A), (np.dot(A, xk) - b))

# 1. MODIFICADO
74. gradiente = np.dot(A, xk) + b

# 2. ORIGINAL
87. valor = (1/2)*np.linalg.norm(np.dot(A, xk) - b, 2)**2

# 2. MODIFICADO
87. valor = 0.5 * np.dot(np.transpose(xk), np.dot(A, xk)) + np.dot(np.transpose(b), xk)
```

Por otro lado, se modificó el archivo main_FOMls.py, ya que, por la naturaleza del problema, necesitamos que la matriz Q sea una matriz simétrica. Para lograr esto, se importó el módulo parametros.py (el utilizado para generar la matriz para el Método del Gradiente). Además, se modificó la manera en la que se calculaba la constante de Lipschitz:

```
# 1. ORIGINAL
36. L = np.linalg.norm(np.matmul(np.transpose(A), A))

# 1. MODIFICADO
43. L = np.max(np.linalg.eigvalsh(A))
```

1.3. Ítem c.

Para este inciso se modificaron las siguientes líneas de código del archivo FOMls_acc.py:

```
# 1. ORIGINAL
79. gradiente = np.matmul(np.transpose(A), (np.dot(A, yaux) - b))

# 1. MODIFICADO
79. gradiente = np.dot(A, yaux) + b

# 2. ORIGINAL
93. valor = (1/2)*np.linalg.norm(np.dot(A, xk) - b, 2)**2

# 2. MODIFICADO
93. valor = 0.5 * np.dot(np.transpose(xk), np.dot(A, xk)) + np.dot(np.transpose(b), xk)
```

Al igual que el inciso anterior, se ocupó la función generar_datos de parametros.py para generar la matriz Q y el vector c (estas se generan para el Método Simple y el Acelerado en el mismo archivo, main_FOMls.py).

1.4. Ítem d.

El condicionamiento de la matriz Q se podía obtener utilizando el siguiente código:

```
condicionamiento = np.linalg.cond(Q)

# Otra forma
mu_max = np.max(np.linalg.eigvalsh(Q))
mu_min = np.min(np.linalg.eigvalsh(Q))

kappa = mu_max/mu_min
```

Este código fue utilizado en `main.py` (donde se ejecuta el Método del Gradiente), a partir de la línea 45; y en `main_FOMIs.py` (donde se ejecutan el Método Simple y Acelerado), a partir de la línea 51 (en este último se ocupa la letra A en lugar de Q , pero fue sólo para mantener la variable que ya estaba en el código previo a la modificación).

2. Problema 2

Para resolver esta pregunta, primero que nada se hicieron cambios en el archivo *main.py*. Este quedó de la siguiente forma:

```
# Modulos creados por usuario
from newton import *
from steepest import *
from parametros import *

# Primero se generan datos para la funcion
# esta informacion es la que pueden cambiar a su antojo
lista_dimensiones = [10, 50, 100, 150, 200, 250, 500, 750, 1000, 2000, 4000]
iteracion_maxima_gradiente = 1000000
epsilon = 0.0001

np.random.seed(31122004)

cabecera = f"{'n':>7} {'cond(Q)':>13} {'iters':>9} {'f(x)':>14} {'||g||':>12} {'lambda':>12}"
print("\n" + cabecera)
print("-" * len(cabecera))

resultados = list()
for dimension_matriz_Q in lista_dimensiones:
    Q, c = generar_datos(dimension_matriz_Q)
    x0 = np.random.rand(dimension_matriz_Q, 1)
    condicionamiento = float(np.linalg.cond(Q))

    iteracion, valor, norma, lambda_ = gradiente(Q, c, x0, epsilon, iteracion_maxima_gradiente)
    iteracion = int(iteracion)
    valor = float(valor[0][0])
    norma = float(norma)
    lambda_ = float(lambda_[0][0])
    resultados.append((dimension_matriz_Q, condicionamiento, iteracion, valor, norma, lambda_))
    print(f"{dimension_matriz_Q:7d} {condicionamiento:13.6e} {iteracion:9d} {valor:14.6e} {norma:12.4e} {lambda_:12.4e}")
```

Imagen 1

Código de archivo *main.py*

Luego, para reducir complejidad, se comentaron las líneas en las cuales se hacía «print» dentro del método del gradiente, salvo aquella que indica el tiempo que se demoró. También, se eliminaron cálculos dentro de los algoritmos proporcionados que eran innecesarios para los objetivos de nuestra pregunta y que complejizaban de más el problema. Finalmente, se dejó de utilizar la función «fmin-bound» y fue reemplazada por un paso óptimo adecuado a la función del problema:

Desarrollo de la fórmula del paso óptimo:

Paso 1:

Sea $f(x) = \frac{1}{2}x^\top Qx + c^\top x$, con $Q = Q^\top$ y $g(x) = \nabla f(x) = Qx + c$. Tomamos una dirección d y buscamos

$$\lambda^* = \min_{\lambda \in \mathbb{R}} f(x + \lambda d).$$

Paso 2:

$$\begin{aligned} f(x + \lambda d) &= \frac{1}{2}(x + \lambda d)^\top Q(x + \lambda d) + c^\top (x + \lambda d) \\ &= \frac{1}{2}x^\top Qx + \lambda x^\top Qd + \frac{1}{2}\lambda^2 d^\top Qd + c^\top x + \lambda c^\top d. \end{aligned}$$

Como Q es simétrica, $x^\top Qd = d^\top Qx$. Además, $g = Qx + c \Rightarrow g^\top d = d^\top Qx + c^\top d$. Por tanto:

$$f(x + \lambda d) = f(x) + \lambda g^\top d + \frac{1}{2}\lambda^2 d^\top Qd.$$

Paso 3:

$$f'(x + \lambda d) = g^\top d + \lambda d^\top Qd = 0 \implies \lambda^* = -\frac{g^\top d}{d^\top Qd}$$

Paso 4: Con $d = -g$:

$$\lambda^* = \frac{g^\top g}{g^\top Qg}.$$

Este es el reemplazo directo de «fminbound» en el contexto de nuestro problema. Al realizar los calculos solicitados, se obtuvo que las matrices de dimension 1000, 2000 y 4000 presentaron una complejidad demasiado grande para ser resueltas. Se puso como límite 1.000.000 de iteraciones para no estar iterando infinitamente en estos casos. Además, por temas de complejidad, se iteró unicamente 100.000 veces para el caso de $n = 4000$.

n	$\text{cond}(Q)$	iters	t [s]	$f(x)$	$\ g\ $	λ
10	3.201068e+00	11	0.000	-7.698389e+00	6.6942e-05	3.5332e+00
50	2.624721e+01	57	0.000	-9.235016e+01	8.9757e-05	4.0321e+00
100	9.477591e+01	217	0.016	-4.407766e+02	9.2507e-05	3.1645e+00
150	2.456424e+02	475	0.031	-1.344946e+03	9.7773e-05	2.6221e+00
200	5.248054e+02	1059	0.087	-2.663708e+03	9.8852e-05	2.2801e+00
250	1.121495e+03	1709	0.231	-3.896310e+03	9.9371e-05	2.0112e+00
500	1.275016e+04	17385	2.572	-3.800767e+04	9.9972e-05	1.5034e+00
750	1.646666e+05	203031	171.999	-1.507557e+05	1.0000e-04	1.3241e+00
1000	2.364100e+06	1000000	1117.801	-7.076785e+05	4.2486e-02	1.3818e+01
2000	4.628583e+10	1000000	2797.308	-1.910342e+07	4.2134e+00	1.3047e+01
4000	1.096408e+11	100000	2201.211	-1.923200e+07	1.3656e+01	1.2968e+01

Tabla 1

Resultados del método del gradiente

3. Problema 3

En el siguiente análisis, se me pide implementar y comparar dos algoritmos de optimización: el Método Simple de Primer Orden (también conocido como Gradiente Descendente) y el Método Acelerado de Nesterov. Para esta tarea, utilizaré los mismos datos de entrada generados en la Pregunta 2, asegurando así una base de comparación consistente. A continuación los resultados obtenidos:

n	$iters$	t [s]	$f(x)$	$\ g\ $	λ
10	20	0.001	-7.698389e+00	9.1745e-05	-9.9943e-01
50	200	0.015	-9.457880e+01	9.6933e-05	-1.0000e+00
100	756	0.051	-4.623841e+02	9.9187e-05	-1.0000e+00
150	1910	0.136	-1.068712e+03	9.9826e-05	-1.0000e+00
200	4763	0.391	-2.311505e+03	9.9980e-05	-1.0000e+00
250	8119	0.734	-4.809756e+03	9.9932e-05	-1.0000e+00
500	95074	12.675	-3.487319e+04	9.9995e-05	-1.0000e+00
750	776210	188.087	-1.623529e+05	9.9999e-05	-1.0000e+00
1000	999999	522.038	-6.815654e+05	2.3679e-01	-1.0000e+00
2000	999999	3507.564	-7.505624e+06	1.9809e+00	-1.0000e+00
4000	99999	1466.668	-6.900662e+06	6.2487e+00	-1.0000e+00

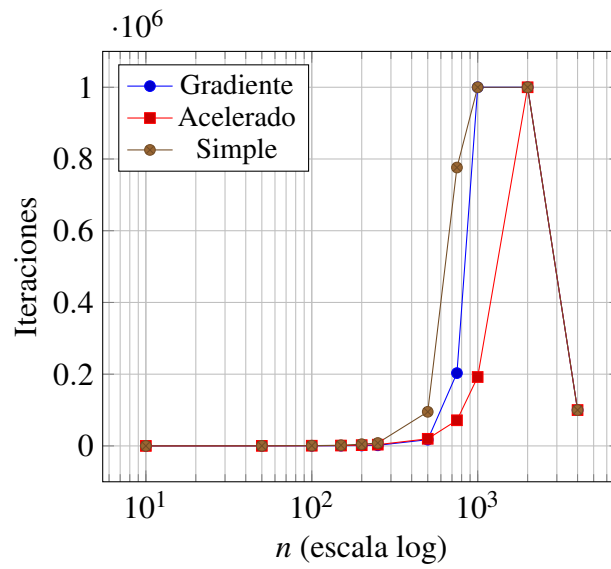
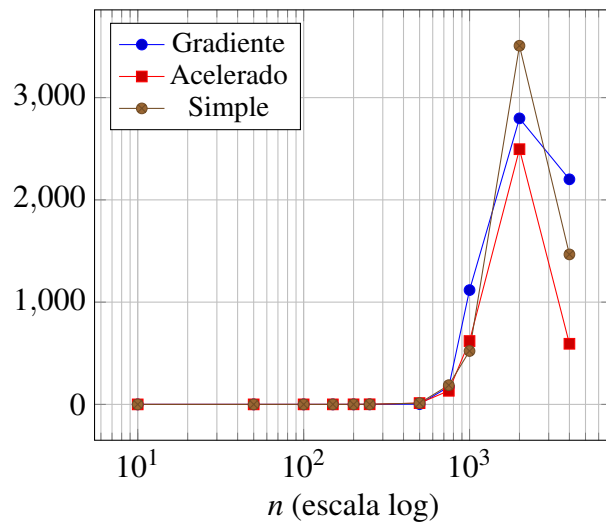
Tabla 2

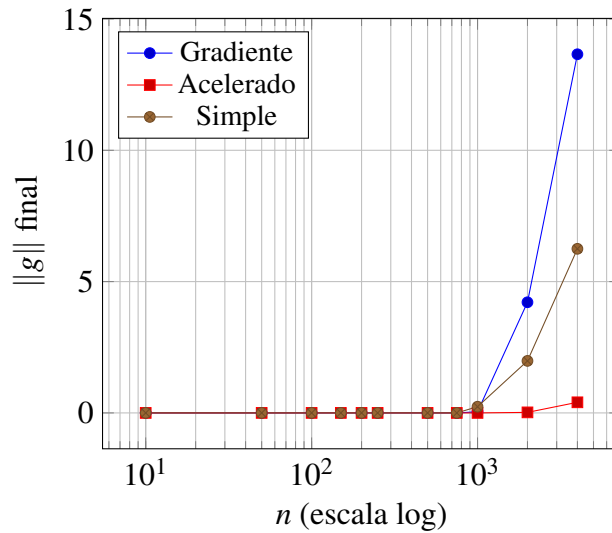
Resultados del método simple de primer orden

n	$iters$	t [s]	$f(x)$	$\ g\ $	λ
10	21	0.003	-7.698389e+00	9.6023e-05	-8.8816e-01
50	156	0.032	-9.457880e+01	9.9287e-05	-9.8706e-01
100	517	0.125	-4.623841e+02	9.9799e-05	-9.9276e-01
150	1013	0.209	-1.068712e+03	9.7056e-05	-9.9909e-01
200	2169	0.607	-2.311505e+03	9.9849e-05	-9.9952e-01
250	3485	1.028	-4.809756e+03	9.9854e-05	-9.9943e-01
500	19798	12.630	-3.487319e+04	9.9670e-05	-9.9998e-01
750	71481	133.063	-1.623529e+05	9.9909e-05	-1.0000e+00
1000	192129	619.485	-7.420235e+05	9.9986e-05	-1.0000e+00
2000	999999	2495.776	-5.688652e+08	2.0534e-02	-1.0000e+00
4000	99999	592.852	-5.770444e+08	4.0425e-01	-1.0000e+00

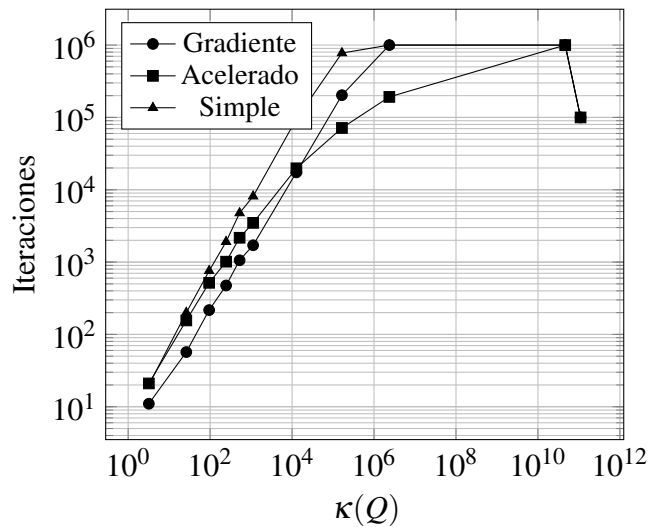
Tabla 3

Resultados del método de primer orden acelerado

**Imagen 2***Iteraciones vs n (eje x log).***Imagen 3***Tiempo de ejecución vs n (eje x log).*

**Imagen 4**

Indicador de convergencia alcanzado vs n (eje x log).

**Imagen 5**

Iteraciones requeridas en función del condicionamiento $\kappa(Q)$.

Con estos datos, se observa que hasta $n \leq 250$ los tres algoritmos alcanzan $\|g\| \approx 10^{-4}$ con pocas iteraciones, pero el «acelerado» paga un «arranque» (por ejemplo, para $n = 10$: se demora 21 iteraciones vs las 11 del gradiente).

Al crecer n y, sobre todo, $\kappa(Q)$ (de 3.2 a 1.10×10^{11}), las diferencias se vuelven marcadas: el «método simple» escala peor en iteraciones (por ejemplo, para $n = 500$: tiene 95,074 iteraciones vs 19,798 del acelerado y 17,385 del gradiente) y deja de alcanzar la tolerancia para $n \geq 1000$. El «gradiente» también se ve fuertemente afectado por el mal condicionamiento y, aunque a $n = 500$ es el más rápido (2.57 s), desde $n = 750$ sus tiempos y el $\|g\|$ final se deterioran (por ejemplo,

$n = 1000$: $\|g\| = 4.25 \times 10^{-2}$ con 1e6 iteraciones). El «acelerado» resulta el más robusto: a $n = 1000$ sí llega a $\|g\| \approx 10^{-4}$ en 619s y, con topes de iteraciones impuestos, mantiene mejores $\|g\|$ finales que los otros dos para $n = 2000$ y 4000 (por ejemplo, para $n = 4000$: $\|g\| = 4.0 \times 10^{-1}$ vs 6.25 del simple y 1.37×10^1 del gradiente), además de tiempos significativamente menores (592 s frente a 1467 s y 2201 s).

En conclusión, «el condicionamiento» es el factor dominante, ya que al crecer $\kappa(Q)$ (peor relación entre curvaturas), los pasos efectivos α_k del gradiente exacto se reducen y la «convergencia lineal» se vuelve extremadamente lenta; el «método simple» (paso casi constante ≈ 1) sufre aún más porque no se adapta a la curvatura. Por otra parte, el «acelerado» amortigua parcialmente ese efecto y conserva buen progreso relativo.

Finalmente, dado que la meta era $\|g\| \leq 10^{-4}$, para $n = 1000, 2000, 4000$ fue necesario fijar topes (1,000,000; 1,000,000; y 100,000 iteraciones, respectivamente): bajo ese régimen de tope, el acelerado entrega las «mejores $\|g\|$ » y los «mejores tiempos» relativos; el gradiente queda intermedio; y el simple resulta el peor en calidad y tiempo en los tamaños grandes.

4. Problema 4

Sea la función:

$$f(x) = \frac{1}{2}x^T Qx + c^T x + \alpha(5 - x_n)^4$$

donde $Q \in \mathbb{R}^{n \times n}$ es simétrica y definida positiva, $c \in \mathbb{R}^n$, α es constante. Además se tiene que:

$$\nabla^2 f(x) = Q + \begin{pmatrix} 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 12\alpha(5 - x_n)^2 \end{pmatrix}$$

Podemos observar que el componente el hessiano presenta una variable x_n que hace que la matriz no sea constante.

4.1. Ítem a.

Podemos argumentar la no existencia de la propiedad de Lipschitz globalmente a través de la demostración por lo absurdo, o sea, sea la definición de Lipschitz para gradientes:

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2 \quad \forall x, y \in B(x^*, y)$$

La desigualdad la transformaremos a:

$$\|\nabla f(x) - \nabla f(y)\|_2 > L\|x - y\|_2$$

Luego, colocandonos en el caso de $x = x^*$, sabemos que el $\nabla f(x) = 0$. Por lo tanto obtenemos:

$$\|-\nabla f(y)\|_2 > L\|x^* - y\|_2$$

Para validar la hipótesis planteada, necesitamos que para dicha igualdad se cumpla, $\|-\nabla f(y)\|_2$ sea un valor cota de la expresión.

Recordemos la expresión del hessiano de la función. Esta esta compuesta por dos componentes, una Q que es definida positiva, y por lo tanto tiene una norma asociada, y una matriz variable, en cual ya mencionaremos que ocurre con sus valores propios.

Dadas las características mencionadas en de la función, podemos obtener la siguiente definición que dice que la constante de Lipschitz es equivalente al mayor valor propio del hessiano:

$$L \approx \max_{x \in D} \|\nabla^2 f(x)\|$$

El problema surge porque la constante de Lipschitz no puede ser única. Esto se debe a que el Hessiano,

en particular el componente matriz variable, contiene un término variable en su diagonal principal: $12\alpha(5 - x_n)^2$. Este término es el único valor propio de la submatriz en conjunto al cero, y dado que su valor depende de x_n , el valor propio también cambia. Como la magnitud de este valor propio varía, no es posible encontrar una única constante de Lipschitz que acote el Hessiano en todo el dominio. Por lo tanto, no se satisface la condición de Lipschitz de forma global.

4.2. Ítem b.

Consideremos que las iteraciones del método siempre están contenidas en una región acotada $D = \{x : \|x\|_2 \leq R\}$, donde R es un valor fijo. Intuitivamente, la condición $\|x\|_2 \leq R$ representa una bola (o un círculo en $\mathbb{R}^2$) en la que se proyecta la función, acotando el vecindario de estudio. Considerando a que la función se comporta de manera no anómala en esta región arbitraria, es posible que exista una constante de Lipschitz L para $\nabla f(x)$ que sea válida en D . Para argumentar esto de forma más rigurosa, consideremos lo siguiente:

$$\|x\|_2 \leq R \implies \|x\| \leq \|x\|_2 \leq R \implies x_n \in [-R, R]$$

Dado que x está en una región acotada, sus componentes también lo están. Esto significa que x_n tiene un rango de valores acotado. Basándonos en el **ítem a.**, sabemos que la constante de Lipschitz es aproximadamente el valor propio más grande del Hessiano:

$$L \approx \max_{x \in D} \left\| \nabla^2 f(x) \right\|_2$$

El Hessiano está compuesto por una matriz definida positiva y otra que es semidefinida positiva que contine de valor propio ceros y uno variable, que es el término $12\alpha(5 - x_n)^2$. Dado que x_n está acotado, el valor de esta expresión también lo está. Si consideramos a R como un valor positivo, lo cual tiene sentido, el término variable alcanza sus valores extremos cuando x_n toma sus valores máximo y mínimo dentro del intervalo $[-R, R]$, o sea, en:

- $12\alpha(5 - (-R))^2 = 12\alpha(5 + R)^2$
- $12\alpha(5 - R)^2$

El mayor de estos dos valores, en conjunto con los obtenidos de la matriz Q , será el que determine la constante de Lipschitz. Dado que la función $12\alpha(5 - x_n)^2$ es continua en el intervalo compacto $[-R, R]$, actuaría el intervalo de forma tal que $[12\alpha(5 - R)^2, 12\alpha(5 + R)^2]$, siempre existirá un valor máximo. Por lo tanto, sí es posible encontrar un valor máximo para la norma del Hessiano en el dominio D , lo que, por definición, garantiza la existencia de una aproximación de la constante de Lipschitz L para $\nabla f(x)$ en dicha región.

4.3. ítem c.

Para demostrar que si α es pequeño, podemos cometer un error grande al estimar R y aun así tener una constante de Lipschitz “razonable”, nos basaremos en los resultados obtenidos en los ítems anteriores.

Sabemos que la constante de Lipschitz para $\nabla f(x)$ en un dominio D está acotada por la norma del Hessiano. Es decir,

$$L \approx \max_{x \in D} \left\| \nabla^2 f(x) \right\|_2$$

Utilizando las propiedades de las normas matriciales, la desigualdad triangular nos permite acotar la norma del Hessiano de la siguiente manera:

$$\left\| \nabla^2 f(x) \right\|_2 = \|Q + D_{\text{var}}\|_2 \leq \|Q\|_2 + \|D_{\text{var}}\|_2$$

Por lo tanto, la constante de Lipschitz queda acotada por la suma de las normas de las matrices, donde $\|Q\|_2$ es una constante fija y $\max_{x \in D} \|D_{\text{var}}\|_2$ es el valor máximo del único valor propio variable del Hessiano:

$$L \leq \|Q\|_2 + \max_{x \in D} \left| 12\alpha(5 - x_n)^2 \right|$$

El problema de que L no sea única surge cuando el dominio D no está acotado, es decir, cuando $R \rightarrow \infty$. Sin embargo, si consideramos un R muy grande y α muy pequeño, estamos analizando un escenario del tipo $0 \cdot \infty$. Para entender el comportamiento del término podemos analizar su límite.

$$\lim_{\alpha \rightarrow 0} \lim_{R \rightarrow \infty} q_n + 12\alpha(5 - x_n)^2$$

Como x_n se mueve en el intervalo $[-R, R]$, la expresión alcanza su valor máximo en los extremos, por lo que analizamos la forma $\alpha(5 + R)^2$, el término se convierte en:

$$L = (q_n + 12 \cdot \lim_{\alpha \rightarrow 0} \alpha \cdot \lim_{R \rightarrow \infty} (5 + R)^2)$$

Esto demuestra que, incluso si R se vuelve muy grande, si α es lo suficientemente pequeño, el término variable del Hessiano no crecerá sin control. Un valor pequeño de α “amortigua” el crecimiento cuadrático de R , resultando en una constante de Lipschitz finita y, por lo tanto, “razonable”.

5. Problema 5

5.1. Ítem a.

El computador estudiado en este caso corresponde al "Macbook Air M2 (2022)" de 8 núcleos (GPU y CPU), cuyo procesador es, como dice el nombre, el chip "Apple M2". Gracias a fuentes disponibles en internet, se logró obtener el dato que dicho computador puede realizar aproximadamente 2.84 TFLOPS (Check-mac, s.f.), sin embargo, no se utilizará esta cifra para mantener la consistencia respecto a la comparación que posteriormente se realizará en el ítem b. El dato está dado en precisión simple (FP32), mientras que el estudio de los supercomputadores del ranking TOP500 se basa en Linpack Benchmark (HPL), el cual mide en doble precisión (FP64) (TOP500, s.f.).

Por lo tanto, se buscó una cifra que si utilizara la medición de TOP500, donde se encontró el repositorio de Jeff Geerling en GitHub, donde permite instalar todo lo necesario para obtener los flops de un computador personal utilizando la escala de TOP500. De esta manera, se obtuvo que el Macbook Air M2 puede realizar aproximadamente 104.68 GFLOPS (Geerling, 2025).

Ya con este dato, se calculará el tiempo en el que se tardaría una iteración de los métodos Newton, Quasi-Newton y Gradiente en segundos, de la siguiente manera:

$$\text{Tiempo de demora de una iteración (s)} = \frac{\text{cantidad de flops requeridos}}{\text{Flop/s que puede realizar el computador}}$$

A continuación se presenta la tabla con los datos obtenidos:

Variables (n)	Newton (s)	Quasi-Newton (s)	Gradiente (s)
50	$1.1941 \cdot 10^{-6}$	$2.3882 \cdot 10^{-8}$	$4.7765 \cdot 10^{-10}$
250	$1.4926 \cdot 10^{-4}$	$5.9706 \cdot 10^{-7}$	$2.3882 \cdot 10^{-9}$
500	$1.1941 \cdot 10^{-3}$	$2.3882 \cdot 10^{-6}$	$4.7765 \cdot 10^{-9}$
1,000	$9.5529 \cdot 10^{-3}$	$9.5529 \cdot 10^{-6}$	$9.5529 \cdot 10^{-9}$
5,000	1.1941	$2.3882 \cdot 10^{-4}$	$4.7765 \cdot 10^{-8}$
20,000	$7.6423 \cdot 10^1$	$3.8212 \cdot 10^{-3}$	$1.9106 \cdot 10^{-7}$
150,000	$3.2241 \cdot 10^4$	$2.1494 \cdot 10^{-1}$	$1.4329 \cdot 10^{-6}$
2,000,000	$7.6423 \cdot 10^7$	$3.8212 \cdot 10^1$	$1.9106 \cdot 10^{-5}$
10,000,000	$9.5529 \cdot 10^9$	$9.5529 \cdot 10^2$	$9.5529 \cdot 10^{-5}$
100,000,000	$9.5529 \cdot 10^{12}$	$9.5529 \cdot 10^4$	$9.5529 \cdot 10^{-4}$

Tabla 4

Tiempo (en segundos) que requiere una iteración de los métodos Newton, Quasi-Newton y Gradiente en un Macbook Air M2 (2022), dependiendo de la cantidad de variables (n).

Analizando exclusivamente los datos presentados en la tabla, notamos que, en caso del Método de Newton, a partir de las 20,000 variables se comienza a demorar un poco más de un minuto en

resolver una iteración, con 150,000 variables se demorara un poco más de 8 horas y con 2,000,000 ya se estaría demorando más de dos años (y para el caso de 100,000,000 de variables se demora un poco más de aproximadamente 302,920 años). Para el Método de Quasi-Newton, se registra el primer tiempo en minutos para el caso de 10,000,000 variables, donde se demora casi 16 minutos, y para las 100,000,000 se demora un poco más de un día. Por último, notamos que para ninguna de las cantidades de variables registradas el Método del Gradiente tarda más de un segundo.

5.2. Ítem b.

Según la lista de TOP500 publicada en junio de 2025, se indica que el supercomputador conocido como "El Capitan" es el más rápido existente en este momento. Este computador cuenta con 11,039,616 núcleos, y su rendimiento máximo corresponde a realizar 1,742 PFlop/s (o 1.742 EFlop/s) (TOP500, 2025). Realizando los mismos cálculos que en el inciso anterior, se obtienen los siguientes resultados:

Variables (n)	Newton (s)	Quasi-Newton (s)	Gradiente (s)
50	$7.1757 \cdot 10^{-14}$	$1.4351 \cdot 10^{-15}$	$2.8703 \cdot 10^{-17}$
250	$8.9696 \cdot 10^{-12}$	$3.5878 \cdot 10^{-14}$	$1.4351 \cdot 10^{-16}$
500	$7.1757 \cdot 10^{-11}$	$1.4351 \cdot 10^{-13}$	$2.8703 \cdot 10^{-16}$
1,000	$5.7405 \cdot 10^{-10}$	$5.7405 \cdot 10^{-13}$	$5.7405 \cdot 10^{-16}$
5,000	$7.1757 \cdot 10^{-8}$	$1.4351 \cdot 10^{-11}$	$2.8703 \cdot 10^{-15}$
20,000	$4.5924 \cdot 10^{-6}$	$2.2962 \cdot 10^{-10}$	$1.1481 \cdot 10^{-14}$
150,000	$1.9374 \cdot 10^{-3}$	$1.2916 \cdot 10^{-8}$	$8.6108 \cdot 10^{-14}$
2,000,000	4.5924	$2.2962 \cdot 10^{-6}$	$1.1481 \cdot 10^{-12}$
10,000,000	$5.7405 \cdot 10^2$	$5.7405 \cdot 10^{-5}$	$5.7405 \cdot 10^{-12}$
100,000,000	$5.7405 \cdot 10^5$	$5.7405 \cdot 10^{-3}$	$5.7405 \cdot 10^{-11}$

Tabla 5

Tiempo (en segundos) que requiere una iteración de los métodos Newton, Quasi-Newton y Gradiente en el supercomputador "El Capitan", dependiendo de la cantidad de variables (n).

Analizando los datos registrados en la tabla, notamos que para una iteración para la cantidad de variables dadas, tanto el Método de Quasi-Newton como el Método del Gradiente, no se demoran más de un segundo. En el caso del Método de Newton, para 2,000,000 variables se demora un poco más de 4 segundos, para 10,000,000 variables se demora aproximadamente 9,6 minutos y para 100,000,000 variables se demorará alrededor de 6 días y medio.

Podemos destacar una gran diferencia entre los tiempos de ejecución del supercomputador "El Capitan" comparado con un computador más de uso diario, como es el Macbook Air M2. Esta abismal diferencia puede deberse a la cantidad de núcleos que posee cada uno, ya que "El Capitan" tiene 11,039,616 núcleos, mientras que el Macbook cuenta con solamente 8. A pesar que ambos

computadores se pueden resolver los problemas con las cantidades de variables propuestas (excepto los casos con 10,000,000 o 100,000,000 en el Macbook, ya que en el primer caso ya se demora más de 302 años, lo cual no es una esperanza de vida razonable para un ser humano), la eficiencia juega un rol muy importante, por lo que no es viable realizar cálculos tan complejos en un computador que no está diseñado para resolver cosas de ese tipo rápidamente.

6. Problema 6

Considerando la siguiente función en n variables:

$$f(x) = \sum_{j=1}^{n-1} |x_j - x_{j+1}| + \|Ax - b\|_2^2 \quad (1)$$

donde $A \in \mathbb{R}^{m \times n}$ y $b \in \mathbb{R}^m$. El primer término de esta función es lo que se conoce como una “medida de variación total”.

6.1. Ítem a.

Como vimos en el taller, la suma de funciones convexas es convexa, por lo que solo hay que demostrar que el primer término $\sum_{j=1}^{n-1} |x_j - x_{j+1}|$ y el segundo término $\|Ax - b\|_2^2$ son convexas.

Primero, analizamos el segundo término $h(x) = \|Ax - b\|_2^2$. Al expandirlo obtenemos:

$$\begin{aligned} h(x) &= (Ax - b)^T (Ax - b) \\ &= x^T (A^T A)x - 2b^T Ax + b^T b \end{aligned}$$

Calculando el gradiente:

$$\nabla h(x) = 2A^T Ax - 2A^T b$$

Y el Hessiano:

$$\nabla^2 h(x) = 2A^T A$$

Sabemos que $A^T A$ siempre es semidefinida positiva, por lo que como el Hessiano es SDP, la función $h(x)$ es convexa. Para el primer término, la función valor absoluto $|x_j - x_{j+1}|$ es convexa para cada j , ya que según Schmidt (2021) el valor absoluto es una función convexa. Por lo tanto, la suma de estas funciones también es convexa.

En conclusión, dado que ambos términos son convexas, la función $f(x)$ es convexa.

6.2. Ítem b.

Para calcular el subdiferencial $\partial f(x)$, podemos dividir la función en dos partes $f(x) = g(x) + h(x)$ y derivar cada una de manera independiente, donde:

$$g(x) = \sum_{j=1}^{n-1} |x_j - x_{j+1}|$$

$$h(x) = \|Ax - b\|_2^2$$

Para el término $h(x)$, como ya calculamos en el apartado anterior, su gradiente es:

$$\nabla h(x) = 2A^T(Ax - b)$$

Para el primer término $g(x)$, aplicaremos la “regla de la cadena” para funciones no diferenciables. Definimos $f(z) = \|z\|_1 = \sum_{j=1}^{n-1} |z_j|$, que sabemos es convexa según Schmidt (2021) al ser una sumatoria de valores absolutos. Además, definimos una matriz $B \in \mathbb{R}^{(n-1) \times n}$ como:

$$B = \begin{pmatrix} 1 & -1 & 0 & \dots & 0 \\ 0 & 1 & -1 & \dots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \dots & 1 & -1 \end{pmatrix}_{(n-1) \times n}$$

Con esta matriz, podemos expresar:

$$Bx = \begin{pmatrix} x_1 - x_2 \\ x_2 - x_3 \\ \vdots \\ x_{n-1} - x_n \end{pmatrix}$$

A partir de esto, podemos ver que $f(Bx) = \|Bx\|_1 = \sum_{j=1}^{n-1} |x_j - x_{j+1}|$, que corresponde exactamente al primer término de la función original $g(x)$.

Para encontrar el subdiferencial de $f(Bx)$, primero determinamos el subdiferencial de $f(z)$ donde $z = Bx$. Para la norma 1, sabemos que el subdiferencial de $|z_j|$ está dado por:

$$\partial |z_j| = \begin{cases} \{1\} & \text{si } z_j > 0 \\ \{-1\} & \text{si } z_j < 0 \\ [-1, 1] & \text{si } z_j = 0 \end{cases}$$

Siguiendo la regla de la cadena, si $p \in \partial f(Bx)$ y $g(x) = f(Bx)$, entonces $B^T p \in \partial g(x)$. Donde

$p \in \mathbb{R}^{n-1}$ y cada componente p_j pertenece al subdiferencial de $|z_j| = |x_j - x_{j+1}|$.

La matriz B^T es:

$$B^T = \begin{pmatrix} 1 & 0 & \dots & 0 \\ -1 & 1 & \dots & 0 \\ 0 & -1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & -1 \end{pmatrix}_{n \times (n-1)}$$

Por lo tanto, el subdiferencial completo de la función original es:

$$\partial f(x) = \{B^T p + 2A^T(Ax - b) : p \in \mathbb{R}^{n-1}, p_j \in \begin{cases} \{1\} & \text{si } x_j - x_{j+1} > 0 \\ \{-1\} & \text{si } x_j - x_{j+1} < 0 \\ [-1, 1] & \text{si } x_j - x_{j+1} = 0 \end{cases}\}$$

6.3. Ítem c.

Para determinar el esfuerzo computacional para calcular un elemento del subgradiente $\partial f(x) = B^T p + 2A^T(Ax - b)$, analizamos el costo de cada operación:

1. Cálculo de $B^T p$:

- $B^T \in \mathbb{R}^{n \times (n-1)}$, $p \in \mathbb{R}^{n-1}$
- B^T tiene a lo más 2 elementos no nulos por fila, con n filas total
- Número de operaciones proporcional a n , por lo tanto $O(n)$

2. Cálculo de Ax :

- $A \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^n$
- Multiplicación matriz-vector estándar
- Número de operaciones proporcional a mn , por lo tanto $O(mn)$

3. Cálculo de $Ax - b$:

- $(Ax) \in \mathbb{R}^m$, $b \in \mathbb{R}^m$
- Resta de m componentes
- Número de operaciones proporcional a m , por lo tanto $O(m)$

4. Cálculo de $A^T(Ax - b)$:

- $A^T \in \mathbb{R}^{n \times m}$, $(Ax - b) \in \mathbb{R}^m$

- Multiplicación matriz-vector estándar
- Número de operaciones proporcional a mn , por lo tanto $O(mn)$

5. Determinación de los valores de p_j :

- Evaluación de $(n - 1)$ diferencias y sus signos
- Número de operaciones proporcional a n , por lo tanto $O(n)$

6. Suma final $B^T p + 2A^T(Ax - b)$:

- Vectores en $\mathbb{R}^n$
- Número de operaciones proporcional a n , por lo tanto $O(n)$

En resumen, el costo computacional viene principalmente de dos multiplicaciones matriz-vector, ambas $O(mn)$, mientras que todas las demás operaciones son de orden menor. Por lo tanto, la complejidad total es $O(mn)$.

7. Problema 7

Sea el siguiente problema de optimización:

$$\begin{aligned} \min(& \max_{i=1,\dots,m} \{\alpha_i^T x - b_i\} + \sum_{j=1}^{n-1} (x_j - x_{j+1})^2) \\ \text{s.a. } & \sum_{j=1}^n x_j \leq 1 \\ & x_j \geq 0, \quad j = 1, \dots, n \end{aligned}$$

Donde $\alpha_i \in \mathbb{R}^n$, $i = 1, \dots, m$, los b_i son escalares, y x es un vector. En conjunto, se tiene que esta función es convexa, con un componente diferenciable y otro no.

7.1. Ítem a.

Determinaremos $\partial f(x)$ en un punto cualquiera del dominio del problema. Dado que la función $f(x)$ no es diferenciable en todos los puntos (debido al término $\max$), debemos encontrar su subdiferencial. La función $f(x)$ es la suma de dos funciones convexas, con una diferenciable y otra no, por lo que su subdiferencial es la suma de los subdiferenciales de cada una.

$$\frac{\partial f(x)}{\partial X} = \frac{\partial}{\partial X} \left(\max_{i=1,\dots,m} \{\alpha_i^T x - b_i\} + \sum_{j=1}^{n-1} (x_j - x_{j+1})^2 \right)$$

Sea:

- Función no diferenciable: $g(x) = \max_{i=1,\dots,m} \{\alpha_i^T x - b_i\}$
- Función diferenciable: $h(x) = \sum_{j=1}^{n-1} (x_j - x_{j+1})^2$

Entonces:

$$\frac{\partial f(x)}{\partial X} = \partial g(x) + \partial h(x)$$

Para ello, sera de gran apoyo diferenciar la expresión anterior respecto a una X_i arbitrario.

7.1.1. Procedimiento subdiferencial $\partial g(x)$

La función $g(x)$ es el máximo de un conjunto de funciones lineales. Esto la hace no diferenciable en los puntos donde el máximo es alcanzado por dos o más funciones. Es por ello que:

- Si en un punto x hay solo un índice k que satisface $\alpha_k^T x - b_k > \alpha_i^T x - b_i$ para todo $i \neq k$, entonces la función es diferenciable en x . En este caso, el subdiferencial es simplemente el gradiente de la función que domina en ese punto, que es el vector α_k .

- Si en un punto x hay dos o más índices que dan el valor máximo, la función no es diferenciable. El subdiferencial es el conjunto los gradientes de esas funciones activas.

Tomemos el caso ideal con tal de simplificar el desarrollo, o sea, que solo tenemos un componente activo, entonces:

$$\begin{aligned}\frac{\partial(g(x))}{\partial X_i} &= \frac{\partial}{\partial X_i}(\max_{i=1,\dots,m} \{\alpha_i^T x - b_i\}) \\ \frac{\partial(g(x))}{\partial X_i} &= \frac{\partial}{\partial X_i}(\{\alpha_i^T x - b_i\}) \\ \frac{\partial(g(x))}{\partial X_i} &= \{\alpha_i\} \\ \frac{\partial(g(x))}{\partial X_i} &= \{\alpha_k\}_{\text{vectoractivos}}\end{aligned}$$

7.1.2. Procedimiento subdiferencial $\partial h(x)$

La función $h(x)$ es una suma de términos cuadráticos y es diferenciable, por lo que generaremos un pequeño truco :

$$\begin{aligned}\frac{\partial(h(x))}{\partial X_i} &= \frac{\partial}{\partial X_i}(\sum_{j=1}^{n-1} (x_j - x_{j+1})^2) \\ \frac{\partial(h(x))}{\partial X_i} &= \frac{\partial}{\partial X_i}(\sum_{j=1, j \neq i}^{n-1} (x_j - x_{j+1})^2 + (x_{i-1} - x_i)^2 + (x_i - x_{i+1})^2) \\ \frac{\partial(h(x))}{\partial X_i} &= \frac{\partial}{\partial X_i}(0 - 2(x_{i-1} - x_i) + 2(x_i - x_{i+1})) \\ \frac{\partial(h(x))}{\partial X_i} &= -2(x_{i-1} - x_i) + 2(x_i - x_{i+1})\end{aligned}$$

Luego, debemos ponernos en casos, ya que i dependera del rango de índices que tome el vector X , por lo que:

- Para $k = 1$:

$$\frac{\partial g}{\partial x_1} = 2(x_1 - x_2)$$

- Para $1 < k < n$:

$$\frac{\partial g}{\partial x_k} = \frac{\partial}{\partial x_k} \left((x_{k-1} - x_k)^2 + (x_k - x_{k+1})^2 \right) = -2(x_{k-1} - x_k) + 2(x_k - x_{k+1})$$

■ Para $k = n$:

$$\frac{\partial g}{\partial x_n} = -2(x_{n-1} - x_n)$$

Por lo que el se tiene la siguiente matriz, correspondiente al $\partial h(x)$, con los respectivos diferenciales por componente:

$$\nabla h(x) = \begin{pmatrix} 2(x_1 - x_2) \\ 2(x_2 - x_3) - 2(x_1 - x_2) \\ \vdots \\ 2(x_{n-1} - x_n) - 2(x_{n-2} - x_{n-1}) \\ -2(x_{n-1} - x_n) \end{pmatrix}$$

7.1.3. Conclusión

Con el anterior procedimiento se obtiene como resultado $\partial f(x)$ en un punto cualquiera del dominio del problema:

$$\frac{\partial f(x)}{\partial X} = \{\alpha_k\} + \begin{pmatrix} 2(x_1 - x_2) \\ 2(x_2 - x_3) - 2(x_1 - x_2) \\ \vdots \\ 2(x_{n-1} - x_n) - 2(x_{n-2} - x_{n-1}) \\ -2(x_{n-1} - x_n) \end{pmatrix}$$

7.2. Ítem b.

Para resolver el problema usando el método de subgradiente, y dado a que el problema tiene algunas restricciones, se debe usar una versión proyectada del método. Para ello se propone el siguiente algoritmo:

Algorithm 1 Método de Subgradiente Proyectado

- 1: **Datos:** $x^0 \in S$, $k = 0$, $\varepsilon > 0$, $S := \{x \in \mathbb{R}^n : \sum_{j=1}^n x_j \leq 1, x_j \geq 0\}$.
 - 2: **while** el criterio de parada no se cumpla **do**:
 - 3: $g^k = \nabla f(x^k)$ donde $\nabla f_1(x^k) \in \partial f_1(x^k)$.
 - 4: $y^{k+1} = x^k - \alpha_k g^k$
 - 5: $x^{k+1} = \Pi_S(y^{k+1})$
 - 6: $k = k + 1$
 - 7: **end while**
-

Para mejorar el entendimiento del anterior algoritmo, describimos los pasos más relevantes a continuación:

- **Paso 1: Inicialización de $(x^0, \varepsilon, \alpha_k)$:**

El algoritmo comienza con un punto de partida factible, x^0 , que debe cumplir todas las restricciones. Se define una tolerancia ε para el criterio de parada y una secuencia de tamaños de paso $\alpha_k > 0$. secuencia que garantizar la convergencia.

- **Paso 3: Cálculo del subgradiente g^k :**

En cada iteración, se calcula un subgradiente g^k de la función objetivo en el punto actual x^k . Debido a que la función objetivo es la suma de una función no diferenciable (f_1) y una diferenciable (f_2), el subgradiente total es la suma del subgradiente del primer término y el gradiente del segundo.

- **Paso 4: Actualización de la posición y^{k+1} :**

Se realiza un paso en la dirección opuesta al subgradiente, con una longitud de α_k , para descender hacia el mínimo de la función. El punto y^{k+1} es un punto "tentativo" que podría no satisfacer las restricciones.

- **Paso 5: Proyección Π :**

Después del paso de subgradiente, el nuevo punto y^{k+1} se proyecta sobre el conjunto factible S para obtener el nuevo punto x^{k+1} . La proyección encuentra el punto en S que está más cerca de y^{k+1} , lo que garantiza que las restricciones siempre se cumplan.

8. Referencias bibliográficas

- Check-Mac (s.f.). *Apple MacBook Air 13" M2 8-GPU (2022) - Technical data and benchmarks*. Recuperado 23 de agosto, 2025, de https://www.check-mac.com/en/product-apple_macbook_air_13_m2_8_gpu_2022
- Geerling, J. (2025). *README de Top500 Benchmark - HPL Linpack*. GitHub. Recuperado 25 de agosto, 2025, de <https://github.com/geerlingguy/top500-benchmark/blob/master/README.md>
- TOP500. (junio de 2025). *JUNE 2025*. Recuperado 24 de agosto, 2025, de <https://top500.org/lists/top500/2025/06/>
- Schmidt, M. (2021). *Convex optimization: Convex sets and functions*. University of British Columbia. Recuperado 2 de septiembre, 2025, de <https://www.cs.ubc.ca/~schmidtm/Courses/440-W21/L4.pdf>
- TOP500. (s.f.). *Frequently Asked Questions*. Recuperado 25 de agosto, 2025, de <https://top500.org/resources/frequently-asked-questions/>